

Branch: 5th Sem, B.Tech. CSE
Duration: 60 minutes

Course: Operating Systems (CSL309)
Max. Marks: 15

Note: Attempt subparts of a question together. State clearly any assumptions and reasoning for the answers

Q1. a) Briefly state the main steps that take place during transitioning from user mode to kernel mode via a system call. [1.5 marks]

Ans: **Refer slides**

b) A process executes following line of C code “if (fork()!=0) {fork();} fork();”. How many NEW processes are created? [1 mark]

Ans: **5 New process (total 6 process)**

c) Consider the following program:

```
main(){
    int fd;
    fd = open("outfile", O_CREAT|O_RDWR); // Line 1
    fork(); // Line 2
    write(fd, "hello", 5);
    exit(0);}
```

Assume that all system calls succeed on a uniprocessor system. Also, assume that a system call cannot be interrupted in the middle of its execution. What will be the contents of the “outfile” file, after all processes have successfully exited? [1 marks, CO1]

Ans: **hellohello**

Fork will create two processes with the same fd pointing to the given file. Two “hello” writes will be observed by the same fd (seek value or pos will be shared). And because of the given assumption, interleaving/mixing of these two writes is not possible.

d) What will be the contents of “outfile” if fork() was called before open() i.e, Line 2 comes before Line 1 in the program (see comments for identifying Line 1 & 2) above. [1 marks]

ans: **hello**

Value of fd will be same for both the process. But open file entry corresponding to both this fd are different. ie. there are two copies of write offset. After open system call, write offset in each of the opened file entry is initialized to zero. And second write will over write the first one.

Q2 In the context of Round Robin (RR) scheduling, answer the following: [3x1 = 3 marks]

(a) What are the major strengths and weaknesses of the RR approach?

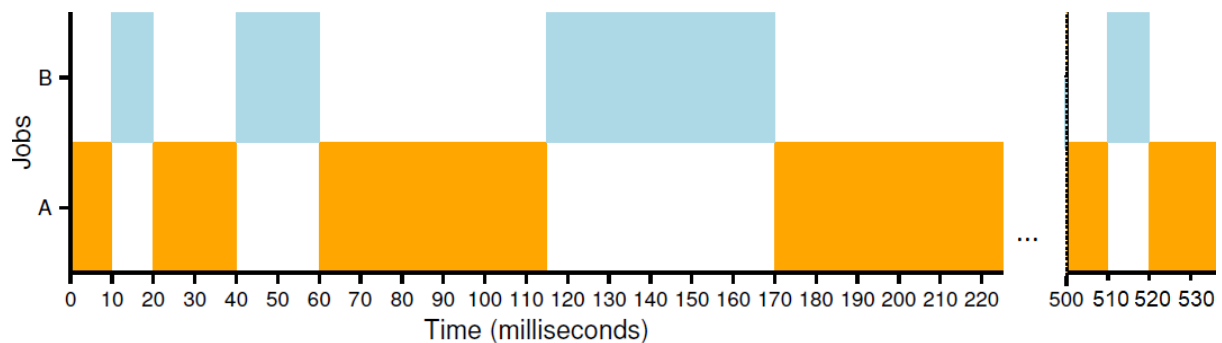
major strengths: improves response time of jobs, as they get scheduled immediately. Also, very simple, and fair across jobs (no starvation for example).

major weaknesses: really bad for turnaround time, as it strings each job out as long as possible. Possibly context-switch overhead too.

(b) How does reducing the time quanta of RR change its behavior/performance? **Improves response time further but, context-switch overhead increases.**

(c) If a random job is picked to run next at each quantum, how would that change its performance (is it better, worse, or quite similar)? **Over shorter time period the performance may be worse but over long span of time it will be similar.**

Q3. Figure below shows the timeline of how two CPU-bound (no I/O) jobs, A and B run over time on an OS using MLFQ (multi-level feedback queue) scheduler. Assume no other jobs are present in the system. Note that in the timeline after 225 till 500, A and B continue to get scheduled alternately with time quantum of 55 milliseconds and after 500 milliseconds, the entire pattern repeats again, indefinitely (until the jobs are done)



(a) How many queues do you think there are in this MLFQ scheduler? **Three queues** [1 marks]

(b) How long is the time slice at the top-most (high priority) and the bottom-most (low priority) queue? **10ms and 55ms** [1 marks]

(d) How often do processes get moved back to the topmost queue? **Every 500 ms** [0.5 marks]

(e) Why does the scheduling policy MLFQ move processes to higher priority levels (i.e., the topmost queue) sometimes? **Most important reason: to avoid starvation. Also, to reassess a job and see if it is behaving "differently" at different points in time** [1 marks]

Q4. An early OS used segmentation to provide some form of primitive virtual memory. The system only had 2 segments (segment 0 for code and a growing heap, segment 1 for a negatively-growing stack). The virtual address space size was only 128 bytes, and there was only 1K of physical memory. Given below is a set of traces from one old program that tells you which virtual address (VA) was accessed (in both hex and decimal forms), and then whether or not the access was valid or not (and hence a segmentation violation). If valid, the physical address (in both hex and decimal) are reported. Oddly enough, programs in this OS were allowed to keep running after memory-access violations, and thus we have a long trace that continues even after such a violation occurred. Here is the trace:

```
VA 0x0000006c (decimal: 108) --> VALID in SEG1: 0x000003ec (decimal: 1004)
VA 0x0000001d (decimal: 29) --> VALID in SEG0: 0x0000021d (decimal: 541)
VA 0x00000050 (decimal: 80) --> SEGMENTATION VIOLATION (SEG1)
VA 0x0000001e (decimal: 30) --> SEGMENTATION VIOLATION (SEG0)
VA 0x00000058 (decimal: 88) --> VALID in SEG1: 0x000003d8 (decimal: 984)
VA 0x00000061 (decimal: 97) --> VALID in SEG1: 0x000003e1 (decimal: 993)
VA 0x00000035 (decimal: 53) --> SEGMENTATION VIOLATION (SEG0)
VA 0x00000021 (decimal: 33) --> SEGMENTATION VIOLATION (SEG0)
VA 0x00000064 (decimal: 100) --> VALID in SEG1: 0x000003e4 (decimal: 996)
VA 0x0000003d (decimal: 61) --> SEGMENTATION VIOLATION (SEG0)
VA 0x0000000c (decimal: 12) --> VALID in SEG0: 0x0000020c (decimal: 524)
```

VA 0x00000005 (decimal: 5) --> VALID in SEG0: 0x00000205 (decimal: 517)
VA 0x0000002f (decimal: 47) --> SEGMENTATION VIOLATION (SEG0)

Now, let's try to recover some values. Note: In case it is not possible to perfectly recover any particular value; provide the best answer you can based on the information above. Briefly explain your answers. From this trace,

a) What was the base and bounds register of segment 0 set to? [1.5 marks]

Base register of segment 0 a) [0.5 marks] : To calculate this, find a valid reference to segment 0.

For example:

VA 0x00000005 (decimal: 5) --> VALID in SEG0: 0x00000205 (decimal: 517)

Here you can see that virtual address 5 translates to physical address 517. Subtracting the offset into segment 0 (5) from 517 gets us 512, which must be the base address (0x400 in hex).

Bounds register of segment 0 [1 marks]: For this, you need to find (in the best case) two references, one that is at virtual address N and is valid, and one that is at address N+1 but is not valid; that will tell us the bound precisely. And hence:

VA 0x0000001d (decimal: 29) --> VALID in SEG0: 0x0000021d (decimal: 541)

VA 0x0000001e (decimal: 30) --> SEGMENTATION VIOLATION (SEG0)

This tells us that any virtual address from 0 through 29 is valid, whereas 30 is not. Hence, the size of segment zero is 30, which we thus conclude is the bound. If you want the physical address instead, it is base plus size, or 542.

b) What was the base and bound registers of segment 1 set to? [2.5 marks]

Base register of segment 1 [1 marks]: Similar to the question above, but remember that segment 1 goes backwards:

VA 0x0000006c (decimal: 108) --> VALID in SEG1: 0x000003ec (decimal: 1004)

Thus, if 108 maps to 1004, we know that 127 (the last valid byte of the virtual address space) would map to 1023 (add 19 to 108 gets us 127; thus add 19 to 1004 to get 1023). Hence, the base (as described in the notes) would be 1024, just beyond the end of where the backward-growing segment lies.

Bounds register of segment 1 [1.5 marks]: The bound here is harder, because it is imprecise.

VA 0x00000050 (decimal: 80) --> SEGMENTATION VIOLATION (SEG1)

VA 0x00000058 (decimal: 88) --> VALID in SEG1: 0x000003d8 (decimal: 984)

As you can see, 80 goes too far backwards, but 88 is fine. With 88, the size must at least allow 88 through 127 to be valid; thus 40 bytes is the minimum segment size. It clearly could also be that 81 is valid, which adds 7 more bytes into the segment size, or 47. Thus, the size of this segment is between 40 and 47, inclusive

----- XXX -----